

Clean Code

Escrevendo código que humanos entendem

Conteúdo

01 Nomenclatura

02 Funções

03 Comentários

04 Formatação

05 Objetos e Estruturas

06 Tratamento de Erros

07 Testes Limpos

08 Classes

09 Princípios e Refatoração

“

Qualquer idiota pode escrever código que um computador entende. Bons programadores escrevem código que humanos entendem.

Martin Fowler, Refactoring (1999)

O que é código limpo?

Legível

Outro desenvolvedor consegue entender sem precisar perguntar ao autor.

Modificável

Mudanças podem ser feitas com confiança e sem cascata de erros.

Testável

Cada unidade pode ser verificada de forma isolada e rápida.

Nomes que revelam intenção

O nome deve dizer por que existe, o que faz e como é usado. Se precisa de comentário para explicar o nome, o nome está errado.

✗ Problemático

```
int d; // dias desde criação

List<int[]> lst;
for (int[] x : lst) {
    if (x[0] == 4)
        result.add(x);
}
```

✓ Limpo

```
int diasDesdeModificacao;

List<int[]> celulasAtivas;
for (int[] celula : celulasAtivas) {
    if (celula[STATUS] == ATIVA)
        resultado.add(celula);
}
```

Pronunciável

Nomes que a equipe consegue dizer em voz alta facilitam discussões.

Buscável

Evite letras únicas fora de laços curtos — impossíveis de encontrar no código.

Sem desinformação

accountList para um conjunto que não é List engana quem lê.

Convenções em Java

Nomenclatura

Classes

Substantivos ou sintagmas nominais

Cliente

ContaBancaria

ProcessadorDePagamento

Métodos

Verbos ou sintagmas verbais

calcularTotal()

salvarCliente()

buscarPorId()

Constantes

UPPER_SNAKE_CASE

TAXA_JUROS_MENSAL

MAX_TENTATIVAS

LIMITE_PADRAO

Booleanos

Prefixo is / has / pode

isAtivo()

hasPermissao()

podeRetirar()

Uma função, uma responsabilidade

Se é possível extrair outra função com nome que não seja mera reformulação da implementação, a função está fazendo mais de uma coisa.

✗ Problemático

```
// faz 3 coisas: validar, calcular,
// persistir
void processarPedido(Pedido p) {
    if (p.getItems().isEmpty())
        throw new RuntimeException();
    BigDecimal total = BigDecimal.ZERO;
    for (Item i : p.getItems())
        total = total.add(i.subtotal());
    p.setTotal(total);
    repository.save(p);
}
```

✓ Limpo

```
void processarPedido(Pedido p) {
    validarPedido(p);
    p.setTotal(calcularTotal(p));
    repository.save(p);
}

void validarPedido(Pedido p) {
    if (p.getItems().isEmpty())
        throw new PedidoVazioException();
}

BigDecimal calcularTotal(Pedido p) {
    return p.getItems().stream()
        .map(Item::subtotal)
        .reduce(ZERO, BigDecimal::add);
}
```

Argumentos e Separação Comando/Consulta

Funções

0

Niládica

Ideal. Sem argumentos.

1

Monádica

Aceitável.

2

Diádica

Requer atenção.

3+

Triádica+

Evitar. Agrupe em objeto.

Separação Comando/Consulta (CQS)

X Misturado

```
// faz algo E retorna resultado
boolean definirAtrib(
    String nome, Object valor) {
    if (existe(nome)) {
        mapa.put(nome, valor);
        return true;
    }
    return false;
}
```

✓ Separado

```
// separados: consulta e comando
boolean existe(String nome) {
    return mapa.containsKey(nome);
}

void definirAtrib(
    String nome, Object valor) {
    mapa.put(nome, valor);
}
```

Quando comentar — e quando não comentar

Comentários

Comentários compensam código que não consegue se expressar sozinho. A resposta certa é limpar o código, não adicionar comentário.

Evitar

- ✗ Comentário que repete o que o código já diz
- ✗ Código comentado (use controle de versão)
- ✗ Comentários de autoria e data de modificação
- ✗ Comentários de posição (===== seção =====)
- ✗ Aviso do tipo "não mexa aqui" sem explicação

Legítimos

- ✓ Licença e cabeçalho de arquivo exigidos pela org.
- ✓ Intenção de algoritmo ou otimização não óbvia
- ✓ Advertência de consequências não triviais
- ✓ TODO com prazo e responsável definidos
- ✓ Javadoc em APIs públicas de bibliotecas

Formatação é comunicação

Formatação

Vertical

Cima para baixo

Funções chamadas aparecem abaixo das chamadoras, como em um jornal.

Próximos no conceito

Código relacionado deve estar verticalmente próximo.

Tamanho do arquivo

Prefira 100 a 500 linhas. Acima de 1.000 linhas é sinal de SRP violado.

Horizontal

```
public class ContaBancaria {  
  
    // (1) constantes  
    private static final BigDecimal  
        LIMITE = new BigDecimal("1000");  
  
    // (2) variáveis de instância  
    private final String numero;  
    private BigDecimal saldo;  
  
    // (3) construtor  
    public ContaBancaria(String num) {  
        this.numero = num;  
        this.saldo = ZERO;  
    }  
  
    // (4) público acima, privado abaixo  
    public void depositar(BigDecimal v) {  
        validar(v);  
        saldo = saldo.add(v);  
    }  
  
    private void validar(BigDecimal v) {  
        if (v.signum() <= 0)  
            throw new IllegalArgumentException();  
    }  
}
```

Objetos vs. Estruturas de dados

Objetos e Estruturas

Objeto

Esconde dados atrás de abstrações. Expõe funções que operam sobre eles. Você sabe o que pode fazer, não como está armazenado.

Estrutura de Dados

Expõe dados diretamente. Sem funções significativas. Útil para transferência entre camadas (DTO). Não misture com objetos de domínio.

Lei de Demeter — fale apenas com seus amigos diretos

✗ Viola a Lei

```
// navega pela estrutura interna
String cidade = funcionario
    .getEndereco()
    .getCidade()
    .getNome();
```

✓ Correto

```
// Funcionario encapsula a navegação
String cidade =
    funcionario.getCidadeResidencia();

// getCidadeResidencia() faz a
// travessia internamente
```

Exceções, não códigos de retorno

Verificar o valor de retorno de cada chamada polui o fluxo e mistura lógica de controle com lógica de negócio.

✗ Código de retorno

```
int r = proc.executar(cmd);  
if (r == ERRO_OCUPADO) {  
    log.error("Ocupado");  
    // tratar...  
} else if (r == ERRO_TIMEOUT) {  
    log.error("Timeout");  
    // tratar...  
}
```

✓ Exceção

```
try {  
    proc.executar(cmd);  
} catch (OcupadoException e) {  
    log.error("Ocupado", e);  
} catch (TimeoutException e) {  
    log.error("Timeout", e);  
}
```

Não retorne null — use Optional<T>

✗ Retorna null

```
// null força verificação no chamador  
Cliente c = buscar(id);  
if (c != null) c.processar();
```

✓ Optional<Cliente>

```
// intenção explícita no tipo  
buscar(id).ifPresent(  
    Cliente::processar);
```

Testes Limpos: F.I.R.S.T.

F

Fast

Rápidos

Testes lentos não são executados com frequência. Unitários devem rodar em milissegundos.

I

Independent

Independentes

Nenhum teste depende de outro. Qualquer um pode ser executado em isolamento.

R

Repeatable

Repetíveis

Mesmo resultado em qualquer ambiente, sem banco de dados, rede ou relógio externo.

S

Self-Valid.

Auto-validáveis

Resultado binário: passa ou falha. Sem inspeção manual de logs ou saída.

T

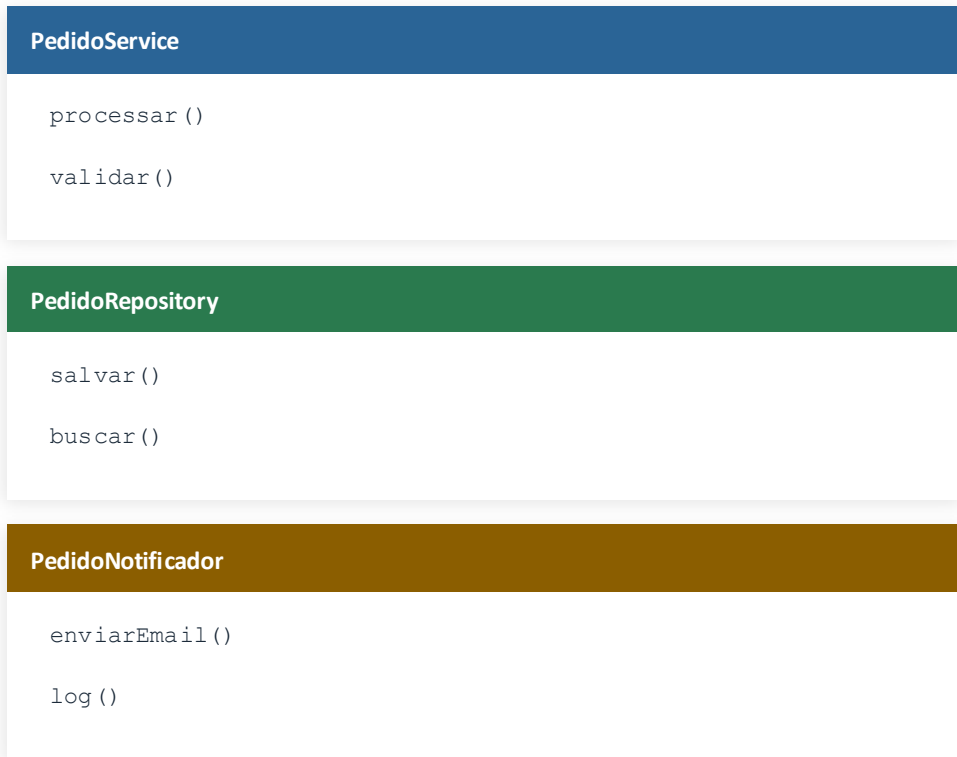
Timely

Oportunos

Escritos junto com o código de produção — ou antes, em TDD.

Princípio da Responsabilidade Única

Uma classe deve ter apenas uma razão para mudar. Uma razão para mudar equivale a uma responsabilidade.



Aberto para Extensão, Fechado para Modificação

Classes

Adicionar um novo comportamento não deve exigir alterar código existente e testado — apenas criar novo código.

✗ Viola OCP — if/else crescente

✓ Conforme com OCP — extensão por nova classe

```
// cada novo formato
// exige editar este método
void exportar(Relatorio r, String f) {
    if (f.equals("PDF")) { ... }
    else if (f.equals("EXCEL")) { ... }
    // adicionar CSV → editar aqui
}
```

```
interface RelatorioExportador {
    void exportar(Relatorio r);
}
// cada formato = nova classe
class ExportadorPdf implements ... { }
class ExportadorExcel implements ... { }
class ExportadorCsv implements ... { }
```

Código existente não é tocado

Reduz risco de regressão ao adicionar funcionalidade.

Testes já escritos permanecem válidos

Cada implementação é testada de forma isolada.

Substituição em runtime

Injeção de dependência decide qual exportador usar.

Princípios Complementares

DRY

Don't Repeat Yourself

Toda vez que lógica existe em mais de um lugar, qualquer correção deve ser feita em todos esses lugares. A solução é abstrair o trecho duplicado em uma única função ou módulo.

KISS

Keep It Simple

Sistemas funcionam melhor quando são mantidos simples. A tendência do desenvolvedor experiente é supercomplicar, antecipando requisitos que podem nunca chegar.

YAGNI

You Aren't Gonna Need It

Não implemente funcionalidade antes de precisar dela. Código que antecipa necessidades hipotéticas adiciona complexidade real hoje para lidar com problemas imaginários amanhã.

“ *Deixe o código mais limpo do que você encontrou.* ”

Regra do Escoteiro · Robert C. Martin

Técnicas mais comuns em Java

Renomear

Variável, método ou classe para nome mais expressivo.

Extrair Interface

Separar o contrato da implementação concreta.

Objeto de Parâmetro

Agrupar argumentos relacionados em uma classe própria.

Extrair Método

Mover trecho de código para método nomeado e reutilizável.

Constante Nomeada

Substituir número mágico por constante UPPER_SNAKE_CASE.

Polimorfismo

Substituir cadeia if/else baseada em tipo por subclasses.

Resumo

1. Nomes revelam intenção: evite abreviações e termos genéricos.
2. Uma função, uma coisa: se der para extrair, extraia.
3. Zero argumento é ideal; booleano como argumento pede divisão da função.
4. Prefira código expressivo a comentários explicativos.
5. Formatação é comunicação: organize o código para leitura de cima para baixo.
6. Objetos escondem dados; estruturas os expõem. Não misture.
7. Use exceções, não códigos de retorno. Não retorne nem passe null.
8. Testes seguem F.I.R.S.T.: rápidos, independentes, repetíveis, auto-validáveis, oportunos.
9. Uma classe, uma razão para mudar (SRP). Coesão alta.
10. DRY, KISS, YAGNI: elimine duplicação, prefira simplicidade, não antecipe.
11. Refatore continuamente com cobertura de testes como rede de segurança.